

Security Scanning for Infrastructure as Code with LLM Integration

1 Executive Summary

To effectively scan Infrastructure as Code security—including IAM configurations—in Terraform and CDK using LLMs, the most impactful approach is a layered architecture that combines traditional rule-based tools (Checkov, tfsec/Trivy, cdk-nag) as a deterministic foundation with LLM-powered contextual reasoning layered on top for complex vulnerability detection, false-positive reduction, and automated remediation. Traditional tools alone suffer from 68–78% false-positive rates and cannot reason about privilege escalation chains or cross-resource dependencies [1], while LLMs without explicit security instructions produce fully secure IaC only 7% of the time [2]—making neither sufficient in isolation.

The most critical finding is that prompt engineering dramatically determines LLM scanning effectiveness: structured prompts with personas, few-shot examples, and chain-of-thought instructions raise detection F1 scores from 58% to 89%, with chain-of-thought approaches achieving up to 553% improvement in vulnerability identification [2, 3]. For IAM specifically, LLMs excel where rule-based tools fail—reasoning about permission combinations that create privilege escalation paths, such as `iam:CreatePolicyVersion` paired with broad resource scope [4].

Two proven integration patterns exist: the MCP Server Pattern, which connects AI assistants to Checkov/Semgrep for delta scanning in CI/CD pipelines [5], and VS Code Custom Agents stored as version-controlled markdown files that embed security expertise directly into developer workflows [6]. For automated remediation, RAG-enhanced frameworks like LLMSecConfig achieve 94% success rates on real-world misconfigurations [7], though post-fix validation remains essential since LLMs may inadvertently break unrelated configurations [8]. Teams should prioritize investing in prompt design, implement scanning at multiple development stages, and use compliance mapping to triage findings by business risk.

2 Introduction

Infrastructure as Code (IaC) has become the standard approach for provisioning cloud resources, with Terraform and AWS CDK leading adoption. However, misconfigurations in IaC templates—particularly in IAM policies—remain a primary cause of cloud security breaches and privilege escalation attacks [9]. Traditional static analysis tools like Checkov, tfsec, and cdk-nag have provided valuable rule-based detection capabilities, but they suffer from high false-positive rates (estimated at 68–78%) and cannot reason about complex policy logic or cross-resource dependencies [1]. The emergence of Large Language Models (LLMs) offers a transformative approach to IaC security scanning, enabling contextual understanding of code intent, nuanced vulnerability detection, and automated remediation that goes far beyond pattern matching [1].

This report examines how to effectively scan IaC security—including IAM configurations—in Terraform and CDK using LLMs. It covers the critical misconfigurations to detect, the landscape of traditional and AI-powered tools, practical integration architectures, prompt engineering techniques, and automated remediation workflows.

3 Critical IAM Misconfigurations in Terraform and CDK

IAM misconfigurations represent one of the most dangerous classes of IaC security issues because they directly control access to cloud resources. Understanding the specific vulnerability patterns is essential for configuring effective scanning, whether using traditional tools or LLMs.

Wildcard permissions are the most prevalent and dangerous pattern. IAM policies using wildcard (*) in the Resource field for actions that support resource-level permissions create overly broad access, enabling cross-service privilege escalation and unauthorized data access [10]. Similarly, using service-level wildcards in actions (e.g., `s3:*`) grants all current and future permissions for that service, violating the principle of least privilege [11]. These patterns are straightforward for rule-based tools to detect but remain surprisingly common in production configurations.

Missing conditions and privilege escalation paths represent a more subtle class of vulnerabilities. Permissions such as `iam:CreatePolicyVersion`, `iam:AttachUserPolicy`, and `sts:AssumeRole` without proper conditions are a primary cause of privilege escalation attacks on cloud platforms [9]. A role that can modify other roles, combined with broad resource scope, can give low-privileged users administrative powers with a single API call [4]. These escalation chains are particularly difficult for traditional tools to detect because they require reasoning about permission combinations rather than individual policy statements.

CDK-specific risks deserve special attention. In CDK, developers with permission to create IAM entities can create roles with permissions beyond their own, enabling privilege escalation that bypasses the developer's own access boundaries [12]. This is a structural risk inherent to how CDK abstracts IAM role creation, making it critical

to implement permission boundaries in CDK deployments.

Traditional tools detect these issues through distinct mechanisms. Checkov uses graph-based cross-resource analysis with over 1,000 built-in policies, including CKV_AWS_356 for wildcard resource detection, and maps findings to CIS, SOC 2, HIPAA, and PCI DSS frameworks [13]. The tfsec tool (now merged into Trivy) uses static HCL analysis with rules like `aws-iam-no-policy-wildcards` rated at HIGH severity [11]. For CDK specifically, `cdk-nag` runs as a CDK Aspect at synthesis time, enforcing rules like `AwsSolutions-IAM4` (restricting AWS managed policies) and `AwsSolutions-IAM5` (wildcard permissions), with support for multiple compliance NagPacks including HIPAA and NIST 800-53 [14].

4 Traditional IaC Security Tools and Their Limitations

The IaC security scanning ecosystem includes several mature tools, each with distinct strengths and trade-offs. Understanding their capabilities and gaps is essential for determining where LLM augmentation adds the most value.

Tool	IaC Support	Key Strengths	Custom Policies	Limitations
Checkov	TF, CFN, K8s, ARM, Docker, Helm	Graph-based analysis, 1000+ policies, broad compliance mapping [13]	Python, YAML	High false-positive rate on complex modules
tfsec/ Trivy	TF (tfsec); TF, CFN, K8s, Docker (Trivy)	Fast Go-based scanning, minimal false positives for Terraform [15]	JSON, YAML, Rego	tfsec deprecated/merged into Trivy in 2024 [16]
Snyk IaC	TF, CFN, K8s, Helm, ARM	IDE/CLI/SCM integration, drift detection, developer-friendly guidance [17]	Proprietary + OPA	Commercial model; proprietary rule engine
cdk-nag	AWS CDK, CloudFormation	Compliance NagPacks (HIPAA, NIST 800-53, PCI DSS 3.2.1) [18]	TypeScript/Python	AWS CDK-only; construct-level checks only
KICS	TF, CFN, K8s, ARM, Docker, Ansible, Pulumi	Broadest format support, query-based architecture [19]	Rego	Lacks graph-based cross-resource analysis

Table: Comparison of Traditional IaC Security Scanning Tools

The fundamental limitation shared by all these tools is their reliance on syntactic pattern matching. According to analysis of traditional SAST approaches, 68–78% of findings are false positives because these tools cannot understand application context, middleware interactions, or the actual intent behind configurations [1]. Modern cloud security weaknesses increasingly stem from logic flaws, abuse of legitimate features, and contextual misconfigurations—problems that regex-based scanning cannot meaningfully address [1]. LLMs fill this gap by providing multi-modal analysis that combines rules, dataflow understanding, and contextual reasoning to dramatically reduce false positives while catching logic-level vulnerabilities.

5 LLM-Powered Security Scanning Tools and Platforms

Several commercial platforms now integrate LLMs directly into IaC security scanning workflows, offering capabilities that extend well beyond traditional rule-based detection.

Snyk’s DeepCode AI integrates Anthropic’s Claude to power automated vulnerability discovery, prioritization, and developer-ready fixes across code, dependencies, containers, and AI-generated artifacts [20]. This represents a full-lifecycle approach where the LLM not only identifies issues but generates contextual remediation code tailored to the specific codebase.

Amazon Q Developer includes 58 Terraform-specific security detectors and 79 CloudFormation detectors covering IAM misconfigurations, encryption gaps, and exposed secrets [21]. These ML-powered detectors go beyond simple pattern matching to understand the security implications of resource configurations in context.

GitHub Copilot Autofix provides AI-powered code suggestions covering approximately 90% of alert types, en-

abling remediation approximately 3x faster on average compared to manual fixes [22]. Its integration with Cod-eQL’s extensibility framework means it can address IaC security findings alongside application code vulnerabilities.

VS Code Custom Agents represent a newer pattern where teams define repository-stored custom Copilot agents under `.github/agents/` that act as AI-powered Terraform security scanners with explicit tool permissions, model selection (e.g., Claude Sonnet), and version-controlled security instructions shared across all developers [6]. This approach democratizes security expertise by embedding it directly into the development environment.

6 Integration Architectures for LLM-Enhanced Scanning

Two primary architectural patterns have emerged for integrating LLMs with IaC security scanning in practice.

The MCP Server Pattern connects AI coding assistants to traditional security tools through a Model Context Protocol server. According to AWS Prescriptive Guidance, this architecture integrates Checkov, Semgrep, and Bandit to enable AI coding assistants like Kiro, Amazon Q Developer, and Cline to automatically scan IaC configurations [5]. Key features include delta scanning of new code segments (reducing computational overhead), isolated security tool environments, and real-time security feedback during code generation [5]. This pattern is particularly powerful because it combines the precision of established rule-based tools with the contextual reasoning of LLMs—the traditional tools handle known patterns while the LLM addresses novel or complex configurations.

The VS Code Custom Agent Pattern takes a more developer-centric approach. Teams define agents as markdown files stored in `.github/agents/` within their repository, specifying the AI model, tool permissions, and detailed security scanning instructions [6]. This pattern excels at making security scanning a natural part of the development workflow rather than a separate CI/CD gate, and it allows security teams to version-control and distribute scanning expertise across the organization.

Both patterns can be integrated into CI/CD pipelines, with the MCP server approach better suited for automated pipeline gates and the custom agent approach optimized for shift-left developer feedback during code authoring.

7 Prompt Engineering for IaC Security Analysis

The effectiveness of LLMs for IaC security scanning depends critically on prompt engineering. Research demonstrates dramatic differences in detection accuracy based on how prompts are structured.

Chart Explanation: This chart illustrates the significant impact of prompt engineering on LLM security detection performance. A general prompt achieves only 58% F1 score, while a guided prompt with step-by-step instructions raises detection to 89% F1 [2]. Chain-of-thought prompting yields a 553% improvement in F1 accuracy for vulnerability identification over baselines [3].

Structured prompt design is the foundation of effective LLM-based scanning. Research on real-world GitHub IaC repositories found that GPT-4o’s F1 score for security smell detection rose from 58% with a general prompt to 89% with a guided prompt containing clear step-by-step instructions [2]. Effective prompts should include five key components: a persona (e.g., “senior cloud security engineer specializing in AWS IAM”), context about the environment, few-shot examples of vulnerability patterns, specific instructions targeting exact vulnerability classes, and explicit output format requirements [23].

Chain-of-thought prompting produces even more dramatic improvements. When applied to vulnerability analysis, chain-of-thought approaches have demonstrated up to 553% higher F1 accuracy for vulnerability identification compared to baselines [3]. This technique is particularly valuable for IAM analysis because it forces the LLM to reason step-by-step through permission combinations, trust relationships, and potential escalation paths rather than making surface-level pattern matches.

Context window management matters significantly for large IaC codebases. Research indicates that LLM performance degrades when critical information appears in the middle of long contexts, so relevant policies should be positioned near the beginning or end of the prompt [24]. For Terraform modules with many resources, this means prioritizing IAM-related configurations and their dependencies in prompt construction.

Where LLMs excel over rule-based tools is in reasoning about privilege escalation chains and implicit dependencies. IAM privilege escalation often hides in complex interactions—a role that can modify other roles, cross-service permission chains, or indirect role inheritance—which rule-based tools cannot trace because they rely on pattern matching [4]. LLMs can evaluate whether a combination of seemingly innocuous permissions (e.g., `iam:AttachUserPolicy` combined with broad resource scope) creates an escalation path, and can assess cross-account trust policies for overly permissive principal configurations [25].

However, important limitations remain. Research found that only about 7% of LLM-generated IaC scripts were fully

LLM Security Detection Accuracy by Prompt Strategy (IaC Vulnerability Detection)

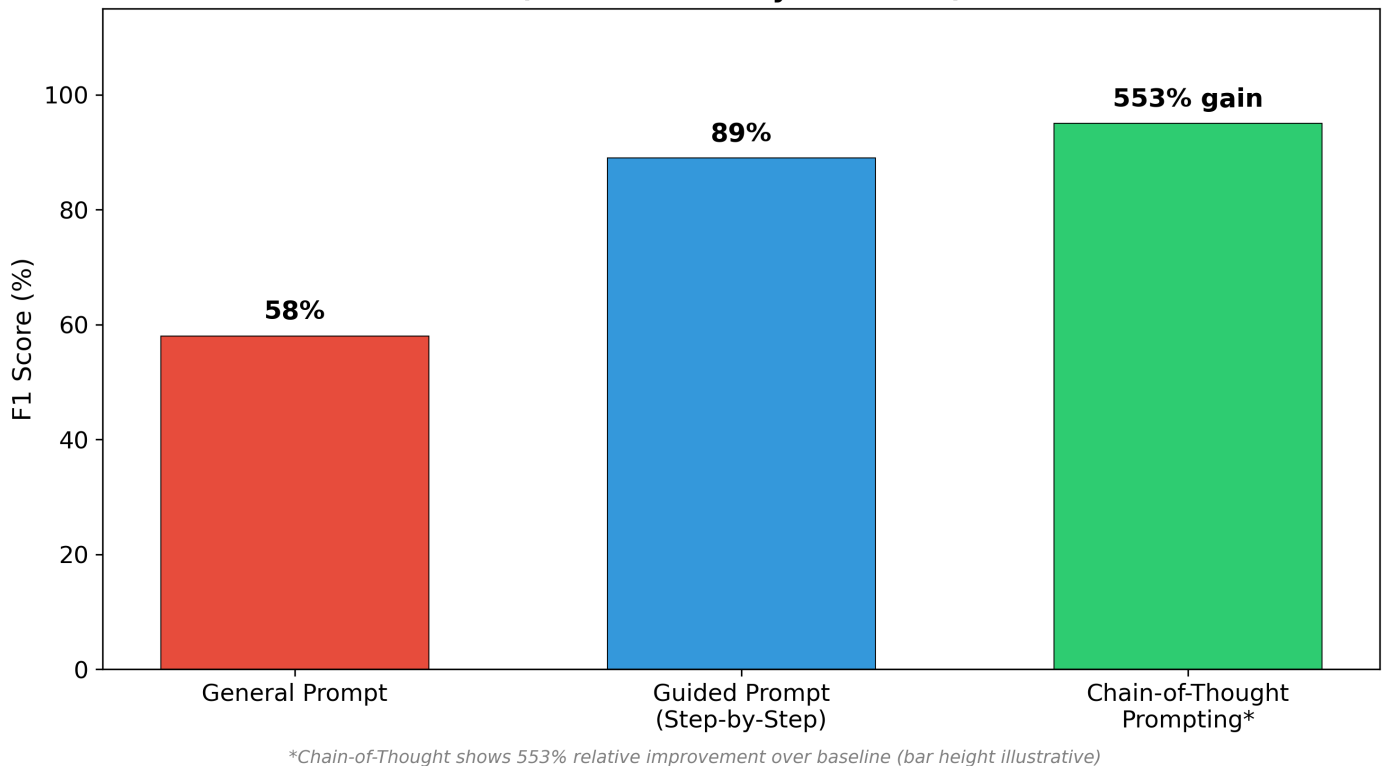


Figure 1: LLM Detection Accuracy by Prompt Strategy

secure without explicit security instructions, and even with security-focused prompts, 44% of GPT outputs still contained security smells [2]. This underscores that LLMs should augment rather than replace dedicated security scanning tools.

8 Automated Remediation and Compliance Mapping

Beyond detection, LLMs enable a paradigm shift from “detect and alert” to “detect and fix” for IaC security issues.

RAG-enhanced remediation represents the most promising approach for automated fixing. The LLMSecConfig framework combines static analysis tools with LLMs using advanced prompting techniques and Retrieval-Augmented Generation to automatically repair security misconfigurations while preserving operational functionality, achieving a 94% success rate on real-world configurations [7]. The workflow follows a detect-decide-act-verify pattern where security events trigger remediation functions that apply fixes via cloud APIs [26].

Commercial auto-fix capabilities are maturing rapidly. Snyk’s Agent Fix product uses an agentic LLM architecture powered by Anthropic’s Claude to auto-fix code vulnerabilities, reducing mean time to remediate by 84% or more [27]. For Terraform specifically, research indicates that structured configuration knowledge injection and explicit security prompt engineering (requiring encryption, public access blocking, access logging) significantly improve the security posture of generated IaC [28].

Compliance mapping is another area where LLMs add substantial value. Snyk IaC supports over 10 compliance standards including CIS Benchmarks, SOC 2, PCI DSS, ISO 27001, and HIPAA, continuously identifying issues mapped to industry benchmarks [29]. LLMs enhance this by converting machine-speed findings into prioritized, developer-ready fixes automatically inside existing workflows [20]. Palo Alto Networks’ Cortex Cloud uses AI-powered cases that group related issues to resolve multiple risks with a single action, reducing manual work by up to 25-fold while prioritizing based on compliance impact and exploitability context [30].

Critical safeguard: While LLMs can generate correct fixes for IaC misconfigurations, research published in Empirical Software Engineering found that they may also delete irrelevant configurations that break the application [8]. This means post-fix validation—running the modified IaC through both the original scanner and integration tests—is essential before applying LLM-generated remediation to production configurations.

9 Practical Implementation Recommendations

For teams looking to implement LLM-enhanced IaC security scanning, a layered approach yields the best results. Start with traditional tools (Checkov for broad coverage, cdk-nag for CDK-specific compliance) as the foundation, then layer LLM capabilities on top for contextual analysis, false-positive reduction, and automated remediation.

The recommended workflow integrates scanning at multiple stages: VS Code custom agents or MCP-connected assistants provide real-time feedback during development [5, 6], pre-commit hooks run fast rule-based checks, CI/CD pipeline gates execute comprehensive scans with LLM-powered triage, and post-deployment monitoring catches drift. When configuring LLM prompts for security analysis, invest in structured prompts with personas, few-shot examples, and chain-of-thought instructions to maximize detection accuracy [2, 23, 3].

Always validate LLM-generated fixes before applying them, maintain human review for critical IAM changes, and use the compliance mapping capabilities to prioritize findings by business risk rather than treating all alerts equally. The combination of deterministic rule-based tools for known patterns and LLM reasoning for novel or complex configurations provides the most robust security posture for IaC deployments.

10 References

1. <https://www.infoworld.com/article/4126765/what-happens-when-you-add-ai-to-sast.html>
2. <https://arxiv.org/html/2602.03648>
3. <https://arxiv.org/abs/2402.17230>
4. <https://hoop.dev/blog/preventing-privilege-escalation-in-infrastructure-as-code-2/>
5. <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/deploy-real-time-coding-security-validation-by-using-an-mcp-server-with-kiro-and-other-coding-assistants.html>
6. <https://techcommunity.microsoft.com/blog/AzureInfrastructureBlog/vs-code-custom-agents-ai-powered-terraform-security-scanning-in-the-ide/4507903>
7. <https://arxiv.org/html/2502.02009>
8. <https://link.springer.com/article/10.1007/s10664-025-10688-0>
9. <https://arxiv.org/html/2304.14540v8>
10. <https://sourcery.ai/vulnerabilities/aws-iam-wildcard-resources>
11. <https://aquasecurity.github.io/tfsec/v1.6.2/checks/aws/iam/no-policy-wildcards/>
12. <https://aws.amazon.com/blogs/devops/secure-cdk-deployments-with-iam-permission-boundaries/>
13. <https://www.env0.com/blog/best-iac-scan-tool-comparing-checkov-vs-tfsec-vs-terrascan>
14. <https://github.com/cdklabs/cdk-nag/issues/703>
15. <https://policyascode.dev/guides/infrastructure-scanning>
16. <https://www.env0.com/blog/best-iac-scan-tool>
17. <https://snyk.io/product/infrastructure-as-code-security/>
18. <https://github.com/cdklabs/cdk-nag/blob/main/README.md>
19. <https://blog.brainboard.co/comprehensive-guide-to-iac-scanning-tools/>
20. <https://snyk.io/news/snyk-embeds-anthropics-claude-to-advance-ai-powered-security-for-software-development/>
21. <https://docs.aws.amazon.com/codeguru/detector-library/terraform/categories/security/>
22. <https://github.com/features/security/code-scanning>
23. <https://crashoverride.com/blog/prompting-llm-security-reviews>
24. Source: wikipedia.org
25. <https://aws.amazon.com/security/security-bulletins/rss/aws-2025-031/>
26. <https://policyascode.dev/guides/policy-remediation>
27. <https://snyk.io/solutions/secure-ai-generated-code/>

28. <https://iee-dataport.org/documents/security-analysis-llm-generated-terraform-configurations-prompt-engineering-effects>
29. <https://snyk.io/blog/automate-cloud-compliance-with-snyk-cloud>
30. <https://www.paloaltonetworks.com/cortex/cloud/demo>